LÖSUNGEN · LEHREREXEMPLAR

W0·4 – Die Zauberschule: Lösungen

Zum Arbeitsblatt `AQ_W0_AB4_Zauberschule_Methoden.docx` · IDE-Level 0.4 und 0.5: siehe `AQ_W0_Akademie_LSG.txt` (Klassen Zauber und ZauberII) · alle Werte mit Java (javac 21) nachgerechnet, alle Methodenköpfe und Compiler-Urteile mit javac geprüft

Teil 1 (DS 4)

Aufgabe 0 – Den Kopf zerlegen

Name **trankPreis** · Rückgabetyp **double** (Kommazahl) · **3** Parameter · 2. Parameter: **boolean** · Signatur: `trankPreis(menge: Ganzzahl, eilig: Wahrheitswert, sorte: Zeichenkette): Kommazahl`.

Aufgabe 1 – Übersetzungsbüro (6 BE)

Nr.	Lösung
1	`public static boolean istMagisch(int zahl)`
2	`public static double flaeche(double breite, double hoehe)`
3	`public static void begruesse(String name)`
4	`public static char letztesZeichen(String s)`
5	`public static int[] zufallsReihung(int laenge, int max)`
6	`public static void startSignal()`
7	anzahlZiffern(n: Ganzzahl): Ganzzahl
8	verbinde(a: Zeichenkette, b: Zeichenkette, trenner: Zeichen): Zeichenkette
9	leere(regal: Reihung von Ganzzahl): nichts
10	enthaeltZeichen(s: Zeichenkette, c: Zeichen): Wahrheitswert
11	halbiert(werte: Reihung von Ganzzahl): Reihung von Kommazahl

Je zwei richtige Zeilen 1 BE (a: 3 BE, b: 3 BE). Typische Fehler: `int a, b`; `Ganzzahl` im Java-Kopf; Rückgabetyp hinten; bei 11 „Reihung von Ganzzahl“ als Rückgabe (es ist `double[]`).

Aufgabe 2 – Vom Auftrag zum Methodenkopf (6 BE)

Nr.	Abi-Signatur	Java-Methodenkopf
1	anzahlUnter(r: Reihung von Ganzzahl, grenze: Ganzzahl): Ganzzahl	`public static int anzahlUnter(int[] r, int grenze)`
2	istVolljaehrig(alter: Ganzzahl): Wahrheitswert	`public static boolean istVolljaehrig(int alter)`
3	ausgeben(r: Reihung von Zeichenkette): nichts	`public static void ausgeben(String[] r)`
4	mittelwert(r: Reihung von Ganzzahl): Kommazahl	`public static double mittelwert(int[] r)`
5	haeufigstesZeichen(s: Zeichenkette): Zeichen	`public static char haeufigstesZeichen(String s)`
6	ohneDuplikate(r: Reihung von Ganzzahl): Reihung von Ganzzahl	`public static int[] ohneDuplikate(int[] r)`

Je Zeile 1 BE; Parameternamen frei. Gesprächsanlass: 3 gibt aus und liefert **nichts** (nicht Zeichenkette!), 4 liefert eine **Kommazahl**.

Aufgabe 3 – Passt der Zauber? (6 BE)

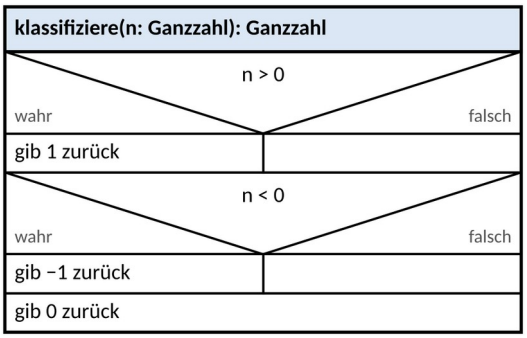
	ok?	Wert / Ausgabe bzw. Grund
A	ja	45
B	nein	Reihenfolge vertauscht: 3 passt nicht zum Parameter `String name`, "Blitz" nicht zu `int stufe`
C	ja	true (heilwert(2) = 30 > 25)
D	nein	Rückgabetyp String passt nicht in eine int-Variable
E	ja	Ausgabe: Licht (Stufe 15)
F	ja	60.0 (Ganzzahl wird automatisch Kommazahl)
G	nein	2.5 ist eine Kommazahl; der Parameter stufe ist int (möglicher Verlust)
H	nein	zu wenige Argumente (istStaerker braucht zwei)

b) Äußerer Aufruf: Parameter **a, b** von istStaerker – Argumente **Zauber.heilwert(2)** (Wert 30) und **25**. Innerer Aufruf: Parameter **stufe** von heilwert – Argument **2** (2 BE). Alle Urteile mit javac geprüft.

Aufgabe 4 – Der Compiler-Detektiv (8 BE)

	akzeptiert?	Grund bzw. Ergebnis
M1	ja	klassifiziere(-7) = -1
M2	nein	„missing return statement“: Für p < 5 endet die Methode ohne return
M3	ja	ersteGerade({3, 7, 4, 8}) = 4 – return beendet Schleife und Methode beim ersten Treffer
M4	nein	„unreachable statement“: println steht hinter return und kann nie ausgeführt werden
M5	ja	melde(-3): **keine Ausgabe** – `return;` beendet die void-Methode vorzeitig

b) Struktogramm (2 BE):



c) z. B. `} else { return "schwach"; }` ergänzen oder `return "schwach";` als letzte Anweisung (1 BE).

Aufgabe 5 – Wer sieht wen? (6 BE)

a) Ausgabe **10 1 30**. Beim Aufruf wird der **Wert** von kraft (10) in den Parameter kraft kopiert; die Methode erhöht nur ihre Kopie (15) und liefert 30. bonus in der Methode ist eine eigene lokale Variable – das bonus des Hauptprogramms bleibt 1 (3 BE).

b) Zeile 7: `i` gilt nur in der for-Schleife. Zeile 8: `quadrat` ist im Schleifenrumpf deklariert und danach unbekannt. Hauptprogramm: `summe` ist lokal in summeBis; richtig wäre `println(summeBis(…));` (3 BE).

Aufgabe 6 – Die Runen der Zauberschule (9 BE)

i	Aufruf von runenwert	Rückgabe	summe
vor der Schleife	–	–	0
0	runenwert('X')	10	10
1	runenwert('V')	5	15

i	Aufruf von runenwert	Rückgabe	summe
2	runenwert('I')	1	16
3	runenwert('I')	1	17
4	runenwert('a')	0	17

Rückgabewert **17** (3 BE). **b)** runenwert wird **3-mal** aufgerufen (für "" läuft die Schleife nicht), z = **30** (2 BE). **c)** Java wertet von links nach rechts aus, Argumente vor dem Aufruf: verdopple(3) → verdopple(1) → verdopple(2); Rückgaben 6, 2, 4; e = 6 + 4 = **10** (2 BE). **d)** z. B. Lesbarkeit (runenSumme zeigt nur das Aufsummieren), einzeln testbar, wiederverwendbar (z. B. für eine Methode „größte Rune“), neue Runen an genau einer Stelle ergänzen (2 BE).

Aufgabe 7 – Level 0.4

```
public static int manaKosten(int stufe, boolean verstaerkt)
public static boolean kannZaubern(int mana, int stufe, boolean verstaerkt)
public static int max3(int a, int b, int c)
public static int runden(double wert)
public static int wuerfelImBereich(int von, int bis)
public static String zauberkette(int stufe)
```

a) Köpfe (3 BE, je zwei Köpfe 1 BE). Rümpfe und häufige Fehler: `AQ_W0_Akademie_LSG.txt`, Klasse Zauber. Nach dem Freischalten gibt das Hauptprogramm u. a. aus: Feuerball Stufe 3 kostet 30 Mana, verstärkt 60; mit 50 Mana normal möglich? true, verstärkt? false; max3(4, 11, 7) = 11; Zauberkette Stufe 5: 1-2-3-4-5.

Freischalten – typische Probleme: nur einen der zwei Blöcke geöffnet (Test läuft, Hauptprogramm-Teil fehlt – unschädlich); nur `/*` oder `*/` gelöscht → Kompilierfehler weit unten; Kopf mit `Ganzzahl` oder ohne Rückgabetyp → Fehlermeldung zeigt auf die **Testzeile** – Lernende sollen dann den Kopf mit der Signatur vergleichen, nicht den Test ändern.

Für Schnelle (Teil 1): römische Zahlen „XIV“ = 14, „MCMXCIV“ = 1994, „XLII“ = 42 (Regel: ist der Wert kleiner als der des nächsten Zeichens, wird er abgezogen); kgV(4, 6) = 12, kgV(12, 18) = 36; tageImMonat(2, 2024) = 29, (2, 1900) = 28, (2, 2000) = 29, (4, 2026) = 30.

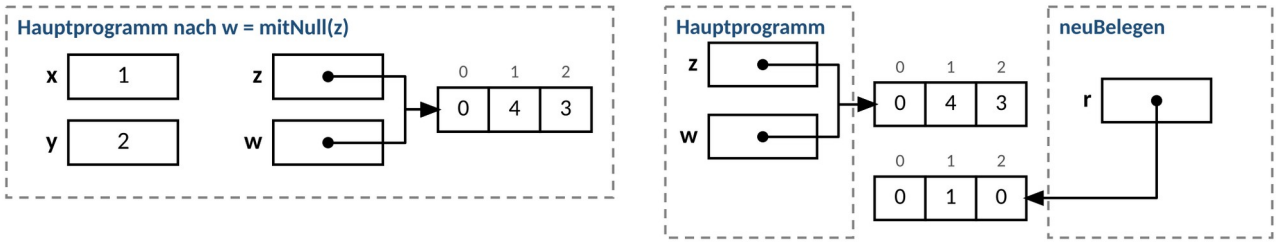
Teil 2 (DS 5)

Aufgabe 8 – Original oder Kopie? (11 BE)

Ausgabe 1	Ausgabe 2	Ausgabe 3	Ausgabe 4
1 2	0 4 9	0 4 9	true

a) 4 BE. Zwischenstände (mit Java protokolliert): nach tauscheInReihung z = {5, 4, 3}; nach mitNull z = w = {0, 4, 3}; neuBelegen ändert nichts; `w[2] = 9` ändert die gemeinsame Reihung.

b) Speicherbilder (3 BE):



c) (4 BE) **tausche:** nicht sichtbar – a und b sind Kopien der Zahlen. **tauscheInReihung:** sichtbar – r verweist auf dieselbe Reihung wie z, getauscht wird der Inhalt. **mitNull:** sichtbar – r[0] = 0 ändert den Inhalt; außerdem liefert sie die **Referenz** auf dieselbe Reihung zurück, w und z sind danach zwei Namen für ein

Objekt (darum true in Ausgabe 4). **neuBelegen:** nicht sichtbar – r wird auf eine neue Reihung umgelenkt; deren Inhalt {0, 1, 0} geht mit dem Methodenende verloren.

Aufgabe 9 – In der Reihung oder neu? (8 BE)

Nr.	Abi-Signatur	Art
1	sortiere(r: Reihung von Ganzzahl): nichts	in-place
2	gespiegelt(r: Reihung von Ganzzahl): Reihung von Ganzzahl	neu
3	nullStattNegativ(r: Reihung von Ganzzahl): nichts	in-place
4	positionenVon(r: Reihung von Ganzzahl, x: Ganzzahl): Reihung von Ganzzahl	neu (Länge zuerst durch Zählen bestimmen)

```
public static void nullStattNegativ(int[] r) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] < 0) {
            r[i] = 0;
        }
    }
}

public static int[] begrenzt(int[] r, int max) {
    int[] neu = new int[r.length];
    for (int i = 0; i < r.length; i++) {
        if (r[i] > max) {
            neu[i] = max;
        } else {
            neu[i] = r[i];
        }
    }
    return neu;
}
```

b) 4 BE (je 2). Geprüft: nullStattNegativ({4, −2, 7, −9}) → r = {4, 0, 7, 0}; begrenzt({3, 12, 7, 20}, 10) liefert {3, 10, 7, 10}, r bleibt {3, 12, 7, 20}; leere Reihung → {}. Typischer Fehler bei begrenzt: im else-Zweig nichts übernehmen (dann stehen Nullen in neu).

Aufgabe 10 – Das vokalreichste Zauberwort (6 BE)

```
public static String bestesWort(String[] woerter) {
    if (woerter.length == 0) {
        return "";
    }
    int bestIndex = 0;
    for (int i = 1; i < woerter.length; i++) {
        if (anzahlVokale(woerter[i]) > anzahlVokale(woerter[bestIndex])) {
            bestIndex = i;
        }
    }
    return woerter[bestIndex];
}
```

a) 4 BE: Leer-Abfrage mit "" (1), Maximum mit Index ab dem ersten Wort (1), Vergleich mit `>` – Gleichstand behält das erste (1), Aufruf der Hilfsmethode und Rückgabe des Wortes (1). Alternative: beste Anzahl in einer Variablen merken. **b)** {"Eis", "Feuer", "Erde"} → **Feuer** (Vokale 2, 3, 2); {"Wind", "Blitz"} → **Wind** (Gleichstand, das erste) (1 BE). **c) 5-mal** – istVokal wird für jedes Zeichen von "Feuer" einmal aufgerufen (1 BE).

Aufgabe 11 – Level 0.5

Musterlösung: `AQ_W0_Akademie_LSG.txt`, Klasse ZauberII. **b)** Ausgaben: Zwischen 10 und 20: **2** (12 und 18) · Index des Minimums: **1** (erste 3) · Zweitgrößter Wert: **18**. Weitere Ausgaben des Hauptprogramms: erhoehteKopie {15, 25, 35} (Original {10, 20, 30}), nach erhoeheAlle {11, 21, 31}, nach fuelle {8, 8, 8}, nach dem Tausch {18, 3, 25, 3, 12}, verschmelze {1, 2, 3, 4, 9, 10}.

Aufgabe 12 – Bug-Hunt (7 BE)

Methode	Fehler	korrigierte Zeile
A	void-Methode gibt einen Wert zurück (Compilerfehler „unexpected return value“)	`public static int max(int a, int b) {`
B	kopiert r[0] bis r[n − 2] statt r[1] bis r[n − 1]	`neu[i] = r[i + 1];`
C	Parameter grenze wird mit 0 überschrieben → zählt alle positiven Werte	Zeile `grenze = 0;` streichen
D	falsche Argumente: anzahl und gold statt preis und anzahl	`return gesamtpreis(preis, anzahl) <= gold;`

b) B: ohneErstes({5, 6, 7}) liefert {5, 6} statt {6, 7}. C: zaehleUeber({3, 8, 12}, 10) liefert 3 statt 1. D: kannKaufen(100, 30, 3) liefert false statt true (90 ≤ 100). Nur A meldet der Compiler (3 BE).

Für Schnelle (Teil 2): rotiereLinks({1, 2, 3, 4, 5}, 2) → {3, 4, 5, 1, 2}; istAnagramm("lager", "regal") = true, ("eis", "sie") = true, ("aab", "abb") = false (Längen vergleichen, dann für jedes Zeichen von s die Anzahlen in s und t vergleichen). Reißverschluss verschmelze({1, 4, 9}, {2, 3, 10}): genommen werden nacheinander 1 (a), 2 (b), 3 (b), 4 (a), 9 (a); die erste Schleife endet mit i = 3, j = 2, k = 5, die Restschleife über b hängt 10 an → {1, 2, 3, 4, 9, 10}.